want to update the critic head. Then we sum the actor and critic losses to get the overall loss. We scale down the critic loss by multiplying by 0.1 because we want the actor to learn faster than the critic. We return the individual losses and the length of the rewards tensor (which indicates how long the episode lasted) to monitor their progress during training.

The way we've set it up here, each worker will update the shared model parameters *asynchronously*, whenever it is done running an episode. We could have designed it such that we wait for all workers to finish running one episode and then sum their gradients together and update the shared parameters synchronously, but this is more complicated, and the asynchronous approach works well in practice.

Put it all together and run it, and you'll get a trained CartPole agent within one minute on a modern computer running on just a few CPU cores. If you plot the loss over time for this, it probably won't be a nice down-trending line like you'd hope because the actor and critic are in competition with one another (figure 5.13). The critic is incentivized to model the returns as best as it can (and the returns depend on what the actor does), but the actor is incentivized to beat the expectations of the critic. If the actor improves faster than the critic, the critic's loss will be high, and vice versa, so there is a somewhat adversarial relationship between the two.

Figure 5.13. The actor and critic have a bit of an adversarial relationship since the actions that the agent take affect the loss of the critic, and the critic makes predictions of state values that get incorporated into the return that affects the training loss of the actor. Hence, the overall loss plot may look chaotic despite the fact that the agent is indeed increasing in performance.

